

Repaso de parciales - Prolog

Observaciones

- En dos parciales de los de abajo se repite un patrón: hacer un predicado/2 principal que sea un generador de naturales (`between(0, inf, _nombre_)`) que llama a un predicado/3 auxiliar donde uno de los parámetros está controlado por ese `_nombre_`, el cual puede ser una cantidad de pasos o algo similar. Dentro del auxiliar se determinan **todas** las restricciones o casos que debe pasar el programa para que **no** se cuelge en un paso puntual de forma infinita (ver [formulasConNSubformulas](#), [caminoDeNPasos](#) y [melodiaDeNTerm](#)). Esto es lo que se conoce como **iterative deepening**.

Recu - 1C25

Ejercicio 1 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. Se deben indicar los patrones de instanciación de todos los predicados auxiliares. No usar macros que colapsen cláusulas, `cut (!)` ni predicados de alto orden, con la única excepción de `not`. **No está permitido el uso de estructuras auxiliares.**

- a) Definir el predicado `unico(+L,-U)` que es verdadero cuando `U` aparece una sola vez en la lista `L`. Resolver sin contar apariciones. Por ejemplo:

```
?- unico([a,b,c,a,a,b,d],U).
U = c ;
U = d ;
false.
```

- b) Definir el predicado `sinRepetidos(+L)` que represente la siguiente propiedad: *una lista `L` hace verdadero a `sinRepetidos` si todos sus elementos son únicos.*

- c) En este inciso vamos a representar a un subconjunto de las fórmulas de lógica proposicional con negación e implicación. Los átomos utilizados serán `neg(P)` e `imp(P,Q)`, con `P` y `Q` fórmulas. Definir el predicado `formula(+VS,-F)` que instancia en `F` todas las fórmulas que usen las variables proposicionales que aparecen en la lista `VS`. Por ejemplo:

<code>?- formula([p,q],F).</code>	<code>F = neg(neg(p));</code>	<code>F = neg(imp(q,q));</code>
<code>F = p;</code>	<code>F = neg(neg(q));</code>	<code>F = imp(p,neg(p));</code>
<code>F = q;</code>	<code>F = imp(p,p);</code>	<code>...</code>
<code>F = neg(p);</code>	<code>F = imp(p,q);</code>	<code>F = imp(neg(q),imp(neg(p),p));</code>
<code>F = neg(q);</code>	<code>...</code>	<code>...</code>

Sugerencia: prestar atención al orden en que se devuelven los resultados. No es necesario que se mantenga el orden del ejemplo, pero sí que se instancien todas las fórmulas.

- d) ¿El predicado anterior es reversible en `F`? Justificar brevemente.

Ej 1

```
unico(L, U) :-
  select(U, L, Resto),      % Saco a U de L, devuelvo Resto, que es la lista L sin U
  not(member(U, Resto)).    % Me fijo si en Resto aparece nuevamente U
```

Ej 2

```
sinRepetidos([]).          % La lista vacía no tiene repetidos
sinRepetidos([H|T]) :-
  not(member(H, T)),        % Veo que H (head) no pertenezca a T (tail)
  sinRepetidos(T).          % Llamo a la función con Tail para la recursión

% Posible solución que veo más rebuscada
% sinRepetidos(L) :- not(noEstaSolo(_, L)).
% noEstaSolo(X, L) :- member(X, L),
%                     not(unico(L, X)).
```

Ej 3

```

formulasConNSubformulas(1, VS, F)           :- member(F, VS).           % Caso base
formulasConNSubformulas(N, VS, neg(F))       :- N > 1,                     % Ej: Si N = 2
                                              N1 is N - 1,                 % Ej: => N1 = 1
                                              formulasConNSubformulas(N1, VS, F). % Ej: formulasConNSubformulas(1,
VS, F). Previamente se corrió formulasConNSubformulas(N, VS, neg(F)), por lo que el F resultante de la primera unifica y
genera neg(F)
formulasConNSubformulas(N, VS, imp(FP, FQ)) :- N > 2,                     % Ej: Si N = 4
                                              N1 is N - 1,                 % => N1 = 3
                                              between(1, N1, N1zq),          % between(1, 3, N1zq). En N1zq
                                              NDer is N1 - N1zq,              % Genero las combinaciones para
                                              formulasConNSubformulas(N1zq, VS, FP), % Llamo a las fórmulas P con la
cantidad de N1zq (paso recursivo)
                                              formulasConNSubformulas(NDer, VS, FQ). % Llamo a las fórmulas Q con la
cantidad de NDer (paso recursivo)
NDer: NDer = 3 - 1, NDer = 3 - 2, NDer = 3 - 3.
formula(VS, F) :- between(0, inf, N),      % Primero prueba N=1, luego N=2...
                  formulasConNSubformulas(N, VS, F). % Genera fórmulas de ese tamaño exacto

```

Parcial - 1C25

Ejercicio 1 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. Se deben indicar los patrones de instanciación de todos los predicados auxiliares. No usar cut (!) ni predicados de alto orden como `setof`, con la única excepción de `not`. No está permitido usar macros que colapsen cláusulas.

- a) Definir el predicado `esRotación(+L,+R)` que es verdadero cuando R es una rotación de L. Una lista R es una rotación de una lista L si contiene los mismos elementos en el mismo orden relativo, pero comenzando desde una posición diferente dentro de L, recorriendo la lista circularmente. Por ejemplo:

```

?- esRotacion([1,2,3,4], [3,4,1,2]).
true.
?- esRotacion([a,b,c], [c,a,b]).
true.
?- esRotacion([a,b,c], [c,b,a]).
false.

```

- b) La *secuencia de Collatz* se construye con la siguiente regla sobre un número natural: si n es par, el siguiente número es $n/2$; si n es impar, el siguiente número es $3n + 1$. La secuencia termina cuando se llega al número 1 (aunque la conjetura dice que siempre se llega a 1, esto no ha sido probado para todos los naturales).

Definir el predicado `collatz(+N,-S)` que instancia en S todos los números pertenecientes a la secuencia de Collatz a partir del número N. Por ejemplo:

```

?- collatz(6,S).
S = 6 ;
S = 3 ;
S = 10 ;
S = 5 ;
S = 16 ;
S = 8 ;
S = 4 ;
S = 2 ;
S = 1 ;
false.

```

- c) ¿El predicado anterior es reversible en S? (pueden asumir que se utilizarán números bajos por lo que la secuencia es siempre finita). Justificar brevemente.
- d) Definir el predicado `collatzMayor(+N,-M)` que es verdadero cuando M es el número más grande de la secuencia de Collatz a partir del número N. Por ejemplo:

```

?- collatzMayor(6,M).
M = 16 ;
false.

```

Ej 1

```

esRotacion(L, R) :-
    length(L, Len), % Chequeo que las longitudes de ambas listas sean iguales
    length(R, Len), % Chequeo que las longitudes de ambas listas sean iguales
    append(L1, L2, L), % Divido a L en 2 partes: L1 y L2. Acá digo que L es la lista resultante de hacer append de L1 y
L2
    append(L2, L1, R). % Chequeo que R sea la lista resultante de hacer append de de L2 y L1 (o sea, la lista L inversa)

```

Ej 2

```
collatz(1, [1]).           % Caso base, cuando se llega a 1 termina.
collatz(N, [N|Resto]) :-   % Caso N par
    N > 1,
    N mod 2 == 0,           % Veo que sea par
    Sig is N // 2,         % Sig = n/2
    collatz(Sig, Resto).    % Paso recursivo
collatz(N, [N|Resto]) :-   % Caso N impar
    N > 1,
    N mod 2 == 1,          % Veo que sea impar
    Sig is 3*N+1,          % Sig = 3*N+1
    collatz(Sig, Resto).    % Paso recursivo
```

Ej 3

```
collatzMayor(N, Max) :-
    collatz(N, L),          % Hago collatz con N, devuelvo la lista L
    maximo_lista(L, Max)    % Busco el máximo en la lista L

maximo_lista([X], X).       % Si la lista tiene un solo elem, ese es el máximo
maximo_lista([H|T], Max) :-
    maximo_lista(T, MaxResto), % Calculo el máximo del tail, lo guardo como MaxResto
    Max is max(H, MaxResto).   % Ahora a Max le doy el valor del máximo entre el head y el máximo del tail (MaxResto)
```

Recu - 2C24

Ejercicio 1 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. Se deben indicar los patrones de instanciación de todos los predicados auxiliares. No usar cut (!) ni predicados de alto orden como `setof`, con la única excepción de `not`.

- a) Implementar el predicado `caminoDesde(+P, -C)` que es verdadero cuando `C` es un camino en un tablero infinito a partir de una posición inicial `P`. Representaremos a los caminos como una lista de posiciones `(X,Y)`, donde `X` e `Y` son números enteros. En cada paso hay cuatro direcciones posibles: arriba, abajo, izquierda y derecha. Por ejemplo

```
?- caminoDesde((0,0),C).
C = [(0, 0)] ;
C = [(0, 0), (0, 1)] ;
C = [(0, 0), (0, -1)] ;
C = [(0, 0), (1, 0)] ;
C = [(0, 0), (-1, 0)] ;
C = [(0, 0), (0, 1), (0, 2)] ;
C = [(0, 0), (0, 1), (0, 0)] ;
C = [(0, 0), (0, 1), (1, 1)] ;
C = [(0, 0), (0, 1), (-1, 1)] ;
C = [(0, 0), (0, -1), (0, 0)] ;
C = [(0, 0), (0, -1), (0, -2)] ;
...
```

- b) Trabajaremos con el conocido problema de optimización de la mochila. Consideremos definidos los hechos `objeto(?Id,?P,?V)` que instancian diversos objetos con identificador `Id`, peso `P` y valor `V`. Implementar el predicado `mochila(+C,-L)` que es verdadero cuando `L` es una lista de identificadores de objetos que al guardarlos en la mochila no supera su capacidad `C`. Por ejemplo, supongamos definidos los siguientes objetos:

```
objeto(1,50,10).
objeto(2,75,15).
objeto(3,60,5).
objeto(4,10,1).
```

Luego, podemos realizar la siguiente consulta:

```
?- mochila(70,L).
L = [] ;
L = [1] ;
L = [1, 4] ;
L = [3] ;
L = [3, 4] ;
L = [4] ;
false.
```

Notar que consideramos a las listas como conjuntos, por lo tanto no se deben devolver soluciones repetidas, es decir, listas que contengan los mismos elementos en distinto orden. El orden de los elementos dentro de la lista es arbitrario.

- c) Analizar la reversibilidad de los parámetros `C` y `L` del predicado anterior.
- d) Definir el predicado `mejorMochila(+C,-L)` que es verdadero cuando `L` es una lista de identificadores de objetos que se pueden guardar en la mochila tal que no superen la capacidad `C` y maximicen el valor. Puede haber más de una solución. Por ejemplo:

```
?- mejorMochila(70,L).
L = [1, 4] ;
false.
```

Ej 1

```
% Primero tengo que definir cómo se comportarán los pasos de acuerdo a los movimientos, defino:

paso((X, Y), (X, Y1)) :- Y1 is Y + 1. % Arriba
paso((X, Y), (X, Y1)) :- Y1 is Y - 1. % Abajo
paso((X, Y), (X1, Y)) :- X1 is X + 1. % Derecha
paso((X, Y), (X1, Y)) :- X1 is X - 1. % Izquierda

% Tengo que controlar la cant. de pasos porque sino puede seguir infinitamente para un lado y nunca encontrar caminos

caminoDeNPasos(P, [P], 0). % El camino tiene 0 pasos, entonces la pos. inicial = camino
caminoDeNPasos(P, [P|T], Pasos) :-
    Pasos > 0,
    PasosT is Pasos - 1, % El LargoT (largo tail) es el largo restante, como recorro solo H, es -1.
    paso(P, Sig), % Sigo al próximo paso
    caminoDeNPasos(Sig, T, PasosT). % Llamo a la función con el nuevo paso, el tail de la lista y la cantidad de
pasos restantes

% Ahora que tengo una forma de controlar los casos donde Pasos = 0 y Pasos > 0, además de que cambio la operación de
movimientos de 'paso', puedo llamar a la original

caminoDesde(P, C) :-
    between(0, inf, Pasos), % Genero pasos (núms naturales) de 0 a inf
    caminoDeNPasos(P, C, Pasos). % Llamo a la auxiliar para que corra infinitamente
```

Ej 2

```

mochila(Capacidad, IDs) :-
    mochila(Capacidad, 0, IDs).

mochila(_, _, []).                                % La lista vacía siempre es válida
mochila(Capacidad, MinID, IDs) :-
    objeto(ID, Peso, _),                          % Declaro el objeto con su ID y Peso, el valor no interesa
    Capacidad >= Peso,                             % La capacidad de la mochila _debe_ ser mayor al peso del obj

    ID >= MinID,                                   % El ID del obj debe ser mayor o igual al minID que actua de "filtro" o iterador
    para que no haya duplicados

    NuevaCapacidad is Capacidad - Peso, % Declaro NuevaCapacidad para hacer el llamado rec
    NextMinID is ID + 1,                  % NextMinID será el próximo ID (como un iterador, filtro) para el llamado rec
    mochila(NuevaCapacidad, NextMinID, RestoIDs). % Llamado rec

```

Ej 3

Acá voy a necesitar definir 2 predicados más para poder definir cuándo una mochila es mejor que otra y para determinar el valor total de una mochila

```

valorMochila([], 0).                                % Si la mochila no tiene elems, su valor es 0
valorMochila([ID|RestoIDs], Valor) :-
    objeto(ID, _, ValorObj),                        % En este caso no interesa la capacidad. Lo declaro para unificar
    valorMochila(RestoIDs, ValorRec),
    Valor is ValorObj + ValorRec.                    % El valor devuelto es la suma del valor del obj + la recursión de valores

existeMochilaMejorValor(Capacidad, MochilaActual) :-
    valorMochila(MochilaActual, ValorOrigMochila), % Calculo el valor de la mochila del predicado
    mochila(Capacidad, MejorMochila),              % Genero otra mochila, que asumo es mejor
    MejorMochila \= MochilaActual,                  % Declaro que ambas son diferentes, sino no tendría sentido
    valorMochila(MejorMochila, ValorMejorMochila), % Calculo el valor de la nueva mochila que supuestamente es mejor
    ValorMejorMochila > ValorOrigMochila.           % El predicado devuelve 'true' si ∃ MejorMochila tq
    ValorMejorMochila > ValorOrigMochila

mejorMochila(Capacidad, Mochila) :-
    mochila(Capacidad, Mochila),
    not(existeMochilaMejorValor(Capacidad, Mochila)).

```


Parcial - 2C25

En este ejercicio trabajaremos con melodías. Se cuenta con el predicado `nota(?N)` que es verdadero cuando `N` es una nota (`do, re, mi, etc.`). Las melodías pueden ser notas o construirse con el término `sec(M1, M2)`, que representa una secuencia de dos melodías. Así, un ejemplo de melodía podría ser `sec(sec(do, fa), la)`, que representa la melodía `do, fa, la`.

- a) Definir el predicado `listaAMelodia(+L, -M)` que es verdadero cuando `M` es una melodía que contiene en orden las notas de la lista `L`. Por ejemplo:

```
?- listaAMelodia([do, re, mi, fa], M).
M = sec(do, sec(re, sec(mi, fa)));
M = sec(do, sec(sec(re, mi), fa));
M = sec(sec(do, re), sec(mi, fa));
M = sec(sec(do, sec(re, mi)), fa);
M = sec(sec(sec(do, re), mi), fa);
false.
```

- b) Definir el predicado `submelodia(+M, -S)` que es verdadero cuando `S` es una subsecuencia de `M`. Una melodía `S` es subsecuencia de una melodía `M` cuando `S` es un subtérmino dentro de la estructura de `M`, obtenido siguiendo algún camino de `sec` sin reordenar ni omitir partes. Es decir, la melodía `sec(do, sec(re, mi))` tiene como subsecuencias a `sec(re, mi)` y `sec(do, sec(re, mi))`, pero no son subsecuencias `sec(do, re)`, `sec(do, mi)`, `do`, `re` ni `mi`. Por ejemplo:

```
?- submelodia(sec(sec(do, sec(re, mi)), fa), S).
S = sec(sec(do, sec(re, mi)), fa);
S = sec(do, sec(re, mi));
S = sec(re, mi);
false.
```

(El predicado es reversible en `S`? Justificar brevemente.)

- c) Definir el predicado `sinSubmelodiasEnComun(+M1, +M2)` que es verdadero cuando `M1` y `M2` no tienen submelodías en común. Por ejemplo:

```
?- sinSubmelodiasEnComun(sec(do, sec(re, mi)), sec(fa, sec(re, mi))).
false.

?- sinSubmelodiasEnComun(sec(do, sec(re, mi)), sec(fa, sec(re, sol))).
true.
```

- d) Definir el predicado `melodia(-M)` que instancia en `M` todas las melodías. No se permite definir `melodia` en función de `listaAMelodia`, debe definirse generando las estructuras en cada paso. Por ejemplo, suponiendo que las únicas notas fueran `do` y `re`:

```
?- melodia(M).
M = do;
M = re;
M = sec(do, do);
M = sec(do, re);
M = sec(re, do);
M = sec(re, re);
M = sec(do, sec(do, do));
M = sec(do, sec(do, re));
...
M = sec(sec(do, re), do);
...
```

Ej 1

```
listaAMelodia([M], M). % Caso base, la lista es solo una melodía
listaAMelodia(L, sec(M1, M2)) :-
    append(L1, L2, L), % Divido L en L1 y L2
    L1 \= [],           % Me aseguro que L1 no sea vacía
    L2 \= [],           % Me aseguro que L2 no sea vacía
    listaAMelodia(L1, M1), % M1 va a contener todas las melodías L1
    listaAMelodia(L2, M2). % M2 va a contener todas las melodías L2

% El input [do, re, mi, fa] devuelve:
% sec(do, sec(re, sec(mi, fa)))
```

Ej 2

```
submelodia(sec(M1, M2), sec(M1, M2)). % Caso base, son exactamente la misma secuencia
submelodia(sec(M1, _), S) :- submelodia(M1, S). % Caso donde S es submelodia de M1
submelodia(sec(_, M2), S) :- submelodia(M2, S). % Caso donde S es submelodia de M2
```

Ej 3

```

sinSubMelodiasEnComun(M1, M2) :-
    not((submelodia(M1, S), submelodia(M2, S))). % Si S es submelodia de ambas, se devolverá 'true'. Con el not,
    devuelve 'false'.

% Va con un paréntesis de más (encerrando ambas submelodías) porque not es not/1, no admite 2 args

```

Ej 4

```

melodia(M) :-
    between(1, inf, N),          % Genero nats desde 1 -> ∞
    melodiaDeTerm(N, M).         % Llamo a la aux para generar infinitamente

melodiaDeTerm(1, M) :- nota(M). % El caso tiene 1 solo término (una nota), devuelvo esa nota
melodiaDeTerm(N, sec(M1, M2)) :-
    N > 1,                      % Me aseguro de no caer en el caso base
    N1 is N-1,                  % Dejo al menos un elem para M2
    between(1, N1, NMelo1)      % Calculo todas las combinaciones para M1
    NMelo2 is N-NMelo1,         % Le doy a NMelo2 el resto de los elementos. Ej: Si NMelo1 = 2 y N = 5 ⇒ NMelo2 = 3
    melodiaDeTerm(NMelo1, M1),  % Llamo a M1 con su N
    melodiaDeTerm(NMelo2, M2).  % Llamo a M2 con su N

```

Parcial - 1C24

- a) Implementar el predicado **generarCapicúas(-L)**, que genera todas las listas capicúas de números naturales (sin incluir al cero). Por ejemplo:

```

?- generarCapicuas(L).
L = [1] ;
L = [2] ;
...
L = [1,3,1] ;
...
L = [2,3,3,2];
...

```

- b) El predicado anterior ¿es reversible? Justificar.

- c) Definir el predicado **tokenizar(+D,+F,-T)**, que es verdadero cuando D representa un diccionario de palabras conocidas (lista de listas de letras), F es una lista de letras sin espacios, y T es la lista de palabras en la que se puede partir la frase. El predicado debe generar todas las posibles maneras de dividir la frase en palabras conocidas. Por ejemplo:

```

?- tokenizar([[a],[a,b],[b,a],[c]], [a,b,a,c,a,b,a], T).
T = [[a],[b,a],[c],[a],[b,a]];
T = [[a],[b,a],[c],[a,b],[a]];
T = [[a,b],[a],[c],[a],[b,a]];
T = [[a,b],[a],[c],[a,b],[a]];
false.

```

- d) Definir el predicado **mayorCantPalabras(+D,+F,-T)** que es verdadero cuando T es una tokenización de la frase F con el diccionario D que tiene la mayor cantidad de palabras. En este inciso no está permitido utilizar estructuras auxiliares.

Ej 1

```

generarCapicuas(L) :-
    between(1, inf, N),          % genero N = 1 -> inf
    listaQueSuma(N, L),          % llamo al generador de listas que suman N
    reverse(L, L).               % verifico que sea capicua

listaQueSuma(0, []).            % si la lista suma 0 => es la vacía
listaQueSuma(N, [H|T]) :-
    N > 0,                       % me aseguro que N > 0
    between(1, N, H),            % genero N = 1 -> inf y los pongo en la pos de H
    Resto is N - H,              % calculo Nresto
    listaQueSuma(Resto, T).      % llamo al tail con nresto

```

`generarCapicuas(L)` no es reversible. Por ejemplo, si se hace `generarCapicuas([1,2])`: `between(1, inf, N)` calcula los N -> `listaQueSuma(N, [1,2])` va a verificar si [1,2] suma 1 -> 'false' -> verifica si suma 2 -> 'false' -> verifica si suma 3 -> 'true' -> `reverse([1,2], [1,2])` devuelve 'false' -> va a seguir calculando si suma 4, 5, 6, etc y siempre va a dar 'false', nunca termina.

Ej 2

```

tokenizar(_, [], []).
tokenizar(Dic, F, [H|T]) :-
    member(H, Dic),              % elijo un elem perteneciente al dic
    append(H, RestoF, F),        % verifico que F empiece con H. declaro RestoF como el tail de F para la rec
    tokenizar(Dic, RestoF, T).   % rec con Dic, el resto de la lista y el tail de tokens

```

Ej 3

```

mayorCantPalabras(D, F, T) :-
    tokenizar(D, F, T)           % candidato T
    length(T, N)                 % calculo su tamaño
    not((
        tokenizar(D, F, T2),     % candidato T2
        length(T2, N2)           % calculo tamaño T2
        N2 > N)).

% Se lee como "Si existe T, no existe T2 tal que (y lo de adentro del not)".

```


Parcial - 2C24

Ejercicio 1 - Programación Lógica

Implementar los predicados respetando en cada caso la instanciación pedida. Los generadores deben cubrir todas las instancias válidas de aquello que generan sin repetir dos veces la misma. Se deben indicar los patrones de instanciación de todos los predicados auxiliares. No usar cut (!) ni predicados de alto orden como `setof`, con la única excepción de `not`.

- a) Definir el predicado `subsecuenciaCreciente(+L,-S)` que es verdadero cuando `S` es una subsecuencia estrictamente creciente de elementos de `L`. Notar que la secuencia respeta el orden de aparición en `L`. Por ejemplo:

```
?- subsecuenciaCreciente([4,8,1,9],S).
S = [4, 8, 9] ;
S = [4, 8] ;
S = [4, 9] ;
S = [4] ;
S = [8, 9] ;
S = [8] ;
S = [1, 9] ;
S = [1] ;
S = [9] ;
S = [].
```

- b) Definir el predicado `subsecuenciaCrecienteMasLarga(+L,-S)` que es verdadero cuando `S` es la subsecuencia estrictamente creciente de mayor longitud de `L`. Puede haber más de un resultado. Por ejemplo:

```
?- subsecuenciaCrecienteMasLarga([5,6,7,2,8,1,2,3,4,5,7],S).
S = [1,2,3,4,5,7] ;
false;

?- subsecuenciaCrecienteMasLarga([5,6,7,2,8,0,2,3,4],S).
S = [5,6,7,8] ;
S = [0,2,3,4] ;
false.
```

- c) Definir el predicado `fibonacci(-X)` que instancia en `X` los números pertenecientes a la secuencia de Fibonacci. Por ejemplo:

```
?- fibonacci(X).
X = 1;
X = 1;
X = 2;
X = 3;
X = 5;
...
```

Ej 1

```
subsecuenciaCreciente(L, S) :-
    subsecuencia(L, S),
    esCreciente(S).

% Lista1 = L1 = [H|T1], Lista2 = L2 = [H|T2]

subsecuencia([], []).
subsecuencia([H|T1], [H|T2]) :-
    subsecuencia(T1, T2).
subsecuencia([_|T1], L2) :-
    subsecuencia(T1, L2).

esCreciente([]).
esCreciente([_]).
esCreciente([A,B|T]) :-
    A < B,
    esCreciente([B|T]).
```

Ej 2

```
subsecuenciaMasLarga(L, S) :-  
    subsecuenciaCreciente(L, S),      % candidato S1  
    length(S, N1),  
    not((  
        subsecuenciaCreciente(L, S2), % candidato S2  
        length(S2, N2),  
        N2 > N1  
    )).  

```

Ej 3

```
fibonacci(X) :-  
    fibonacci_aux(0, 1, X).  
  
fibonacci_aux(_, Act, Act).  
fibonacci_aux(Ant, Act, X) :-  
    Siguiente is Ant + Act,  
    fibonacci_aux(Act, Siguiente, X).  

```

`fibonnaci(X)` no es reversible porque si se le pasa un X, sería algo así: `fibonnaci(5)` -> Prolog genera 1, 1, 2, 3, 5 -> Devuelve 'true' -> Si se le pide más soluciones (con ;), va a seguir generando números y se va a colgar porque los números no paran de crecer y no va a encontrar otra vez al 5. Lo mismo pasa con todos los números